MACQUARIE University

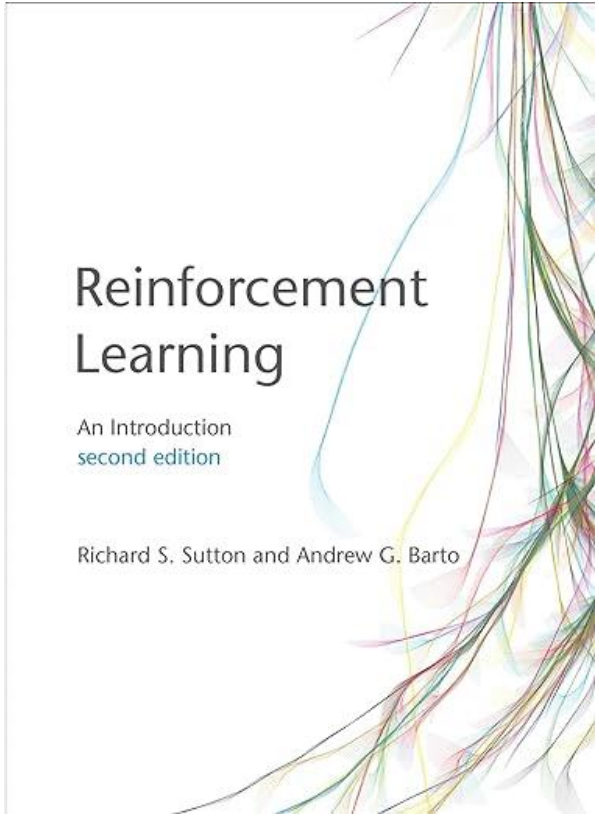
COMP8221 Advanced Machine Learning

Introduction to Deep Reinforcement Learning

Dr Zitai Qiu, School of Computing, zitai.qiu@mq.edu.au



Recommended Textbook



http://incompleteideas.net/book/RLbook2020.pdf



Outline



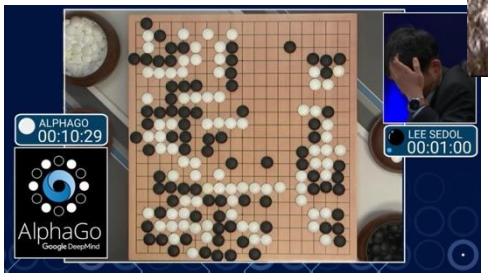
MACQUARIE
University

- Markov Decision Processes
- Monte Carlo Return
- Value Functions
- Q-learning
- Overestimation Bias

What can RL do?

- AlphaGo
- Robotics
- Autonomous driving
- Video games
- …







Types of RL algorithms

MACQUARIE University

- Policy gradients:
  - REINFORCE
- Value-based:
  - DQN
- Actor-critic:
  - DDPG
  - Soft Actor Critic
- Model-based RL:
  - Dyna

Markov Decision Processes

MDPs are a classical formalization of sequential decision making, and a mathematical idealized form of the reinforcement learning problem.

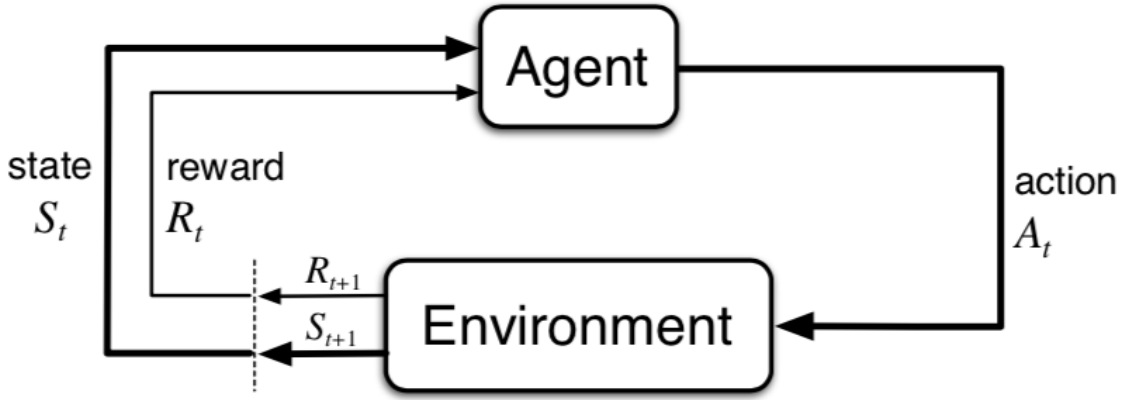


Figure 3.1: The agent–environment interaction in a Markov decision process.

Markov Decision Processes

MACQUARIE University

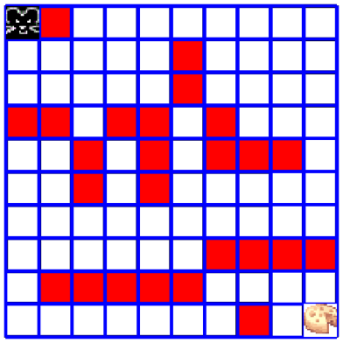
Define Markov Decision Processes as $(S, A, r, p, \rho, \gamma)$:

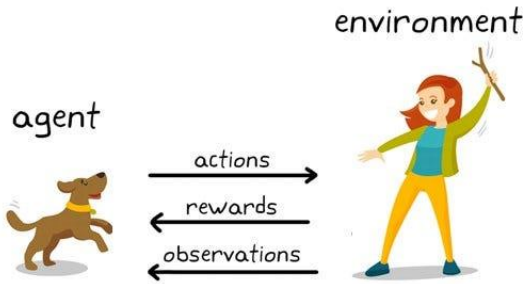
- S: continuous multi-dimensional state space / discrete state space
- A: continuous multi-dimensional action space / discrete action space
- r: bounded reward function $r(s, a)$ maps state action pairs (s, a) into reward received by the agent by taking the action a while in state s
- p: $p(s'|s, a)$ is the probability of transitioning into the next state s' by taking action a in state s
- ρ: initial state distribution
- γ: discount factor [0, 1]

Markov Decision Processes

Actions influence not just immediate rewards, but also subsequent states, and through future rewards.

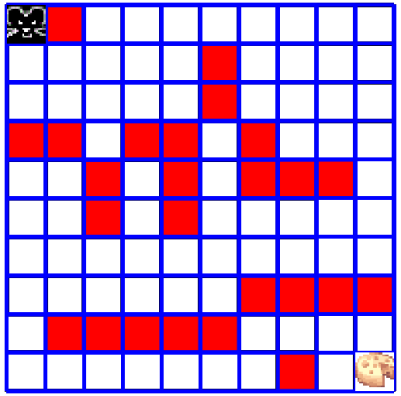
Examples of MDPs: what are the actions? What are the states? Rewards?





Markov Decision Processes

MDPs:



S: possible location of the mouse in the maze.

A (discrete) : Up, Down, Left and Right

r : -1 for each action, -2 for hitting the obstacles,
+10 for reaching the cheese

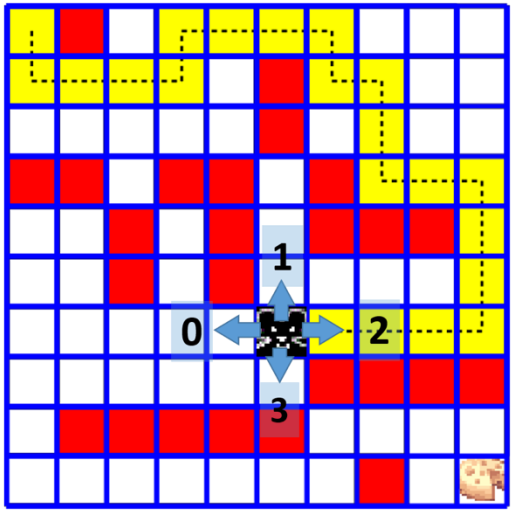
p: $p(s'|s,a) = 1$ for all state s and next state s' as it is a deterministic environment

ρ: any start location of the mouse in the maze

γ: 0.9

Markov Decision Processes: Policy

- Let $\pi = \pi(a|s), s \in S, a \in A$ denote the **policy**. The policy maps from the state space S to a probability distribution over the action space A.



- $\pi(a_0|s) = 0.4$
- $\pi(a_1|s) = 0.1$
- $\pi(a_2|s) = 0.2$
- $\pi(a_3|s) = 0.3$

Probabilities sum up to 1

Markov Decision Processes

Let $\tau_\pi = (s_0, a_0, r_0, s_1, a_1, r_1, \dots)$ as a sample trajectory/episode following policy π.

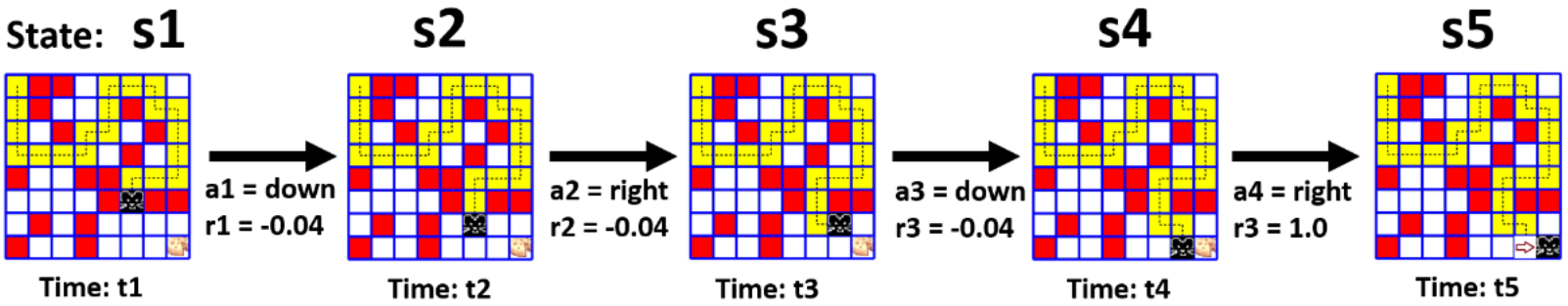
To collect the trajectory, we can describe the process as follows:



1. at time t, the agent is currently in state s_t
2. takes action a_t according to the policy π
3. receives reward $r_t = r(s_t, a_t)$
4. transit to the next state s_{t+1} at time $t+1$ according to $p(\cdot\,|s_t, a_t)$

Markov Decision Processes

Take the maze as example



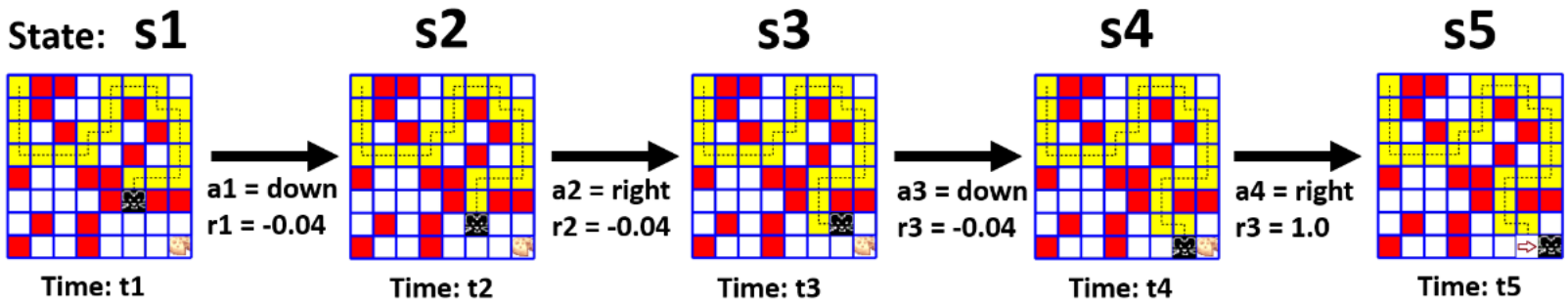
Monte Carlo Return

Take $\tau_\pi = (s_0, a_0, r_0, s_1, a_1, r_1, \dots)$ as a sample trajectory/episode following policy π.

The Return $\boldsymbol{G_t}$ of the trajectory is defined as the total discounted rewards from time t:

$$G_t = r_t + \gamma \cdot r_{t+1} + \gamma^2 \cdot r_{t+2} + \cdots$$

Monte Carlo Return



Assume $\gamma = 0.9$, calculate the return for this single trajectory.

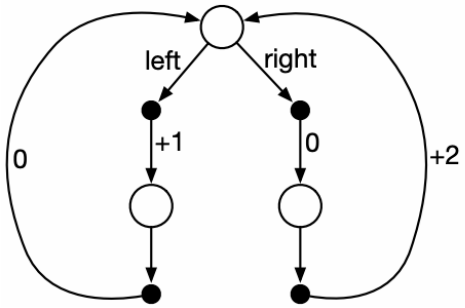
Why γ?

Why discounting with γ?

- Mathematically convenient to discount rewards
- Avoids infinite returns in cyclic Markov processes
- Weight more for immediate reward / more recent rewards
- …

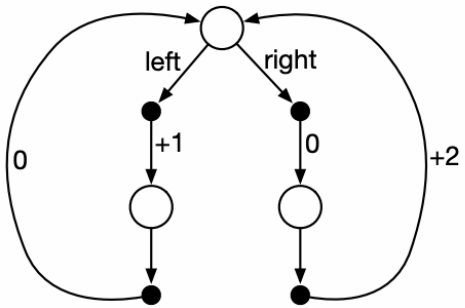
Why discounting with γ?

Exercise 3.22 Consider the continuing MDP shown to the right. The only decision to be made is that in the top state, where two actions are available, left and right. The numbers show the rewards that are received deterministically after each action. There are exactly two deterministic policies, π_{left} and π_{right}. What policy is optimal if $\gamma = 0$? If $\gamma = 0.9$? If $\gamma = 0.5$? □



Why discounting with γ?

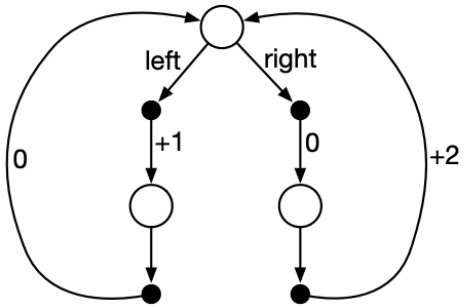
Exercise 3.22 Consider the continuing MDP shown to the right. The only decision to be made is that in the top state, where two actions are available, left and right. The numbers show the rewards that are received deterministically after each action. There are exactly two deterministic policies, π_{left} and π_{right}. What policy is optimal if $\gamma = 0$? If $\gamma = 0.9$? If $\gamma = 0.5$? □



$$\gamma = 0, G_{left} = 1 + 0 * 0 + 0^2 * 1 + 0^3 * 0 + \ ... = 1$$
$$G_{right} = 0 + 0 * 2 + 0^2 * 0 + 0^3 * 2 + \ ... = 0$$
$$\gamma = 0.9, G_{left} = 1 + 0.9 * 0 + 0.9^2 * 1 + 0.9^3 * 0 + \ ... = 1 + 0.81 + \cdots$$
$$G_{right} = 0 + 0.9 * 2 + 0.9^2 * 0 + 0.9^3 * 2 + \ ... = 1.8 + 1.458 + \ ...$$
$$\gamma = 0.5, G_{left} = 1 + 0.5 * 0 + 0.5^2 * 1 + 0.5^3 * 0 + \ ... = 1 + 0.25 + \cdots$$
$$G_{right} = 0 + 0.5 * 2 + 0.5^2 * 0 + 0.5^3 * 2 + \ ... = 1 + 0.25 + \ ...$$

Why discounting with γ?

Exercise 3.22 Consider the continuing MDP shown to the right. The only decision to be made is that in the top state, where two actions are available, left and right. The numbers show the rewards that are received deterministically after each action. There are exactly two deterministic policies, π_{left} and π_{right}. What policy is optimal if $\gamma = 0$? If $\gamma = 0.9$? If $\gamma = 0.5$? □



$\gamma = 0, \pi_{left}$ is optimal.
$\gamma = 0.9, \pi_{right}$ is optimal.
$\gamma = 0.5$, either is optimal.

Value Functions

State value function: **expected discounted return** for policy π beginning in state s

$$V_\pi(s) = E_{a_t \sim \pi}[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \,|s_0 = s]$$
$$= E_{a_t \sim \pi}[G_0 | s_0 = s]$$

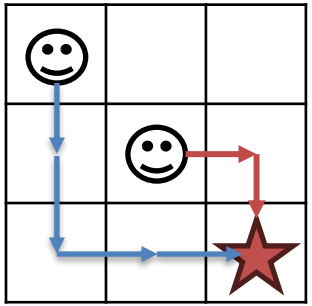
accumulated rewards

Action value function: **expected discounted return** for policy starting in state s and initially taking action a but then following policy π:

$$Q_\pi(s, a) = E_{a_t \sim \pi}[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \,|s_0 = s, a_0 = a]$$
$$= E_{a_t \sim \pi}[G_0 | s_0 = s, a_0 = a]$$

Value Functions

Reward = +1 when reaching the goal; 0 otherwise.
Action: Up, Down, Left, Right



State-Value Function $V_\pi(s)$

"How good is it to be in a particular state?"

State s="middle cell"

Suppose under policy π, it takes 2 steps to reach the goal (red path):

$$V_\pi(\text{s="middle cell"}) = 0 + \gamma \cdot 1 = \gamma, \quad \gamma \in [0, 1]$$

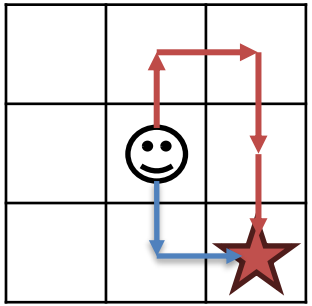
In state s="top left", it takes 4 steps to reach the goal (blue path):

$$V_\pi(\text{s="top left"}) = 0 + \gamma \cdot 0 + \gamma^2 \cdot 0 + \gamma^3 \cdot 1 = \gamma^3, \quad \gamma \in [0, 1]$$

These values reflect how close each state is to the goal **on average under policy π**

Value Functions

Reward = +1 when reaching the goal; 0 otherwise.
Action: Up, Down, Left, Right



State-Action Value Function Q_π(s, a)

"How good is it to take action a in state s?"

State s="middle cell", a = "Up", and then follow policy π (red path).

It takes 4 steps to reach the goal, for example:

$$Q_\pi(\text{s="middle cell", a = "Up"}) = 0 + \gamma \cdot 0 + \gamma^2 \cdot 0 + \gamma^3 \cdot 1 = \gamma^3, \quad \gamma \in [0, 1]$$

State s="middle cell", a = "Down", and then follow the same policy π (blue path).

$$Q_\pi(\text{s="middle cell", a = "Down"}) = 0 + \gamma \cdot 1 = \gamma, \quad \gamma \in [0, 1]$$

This means "Down" is a **better action** in that state than "Up" under policy π.

Value Functions

State value function: $V_\pi(s)$ → How good is state s following policy π

Action value function: $Q_\pi(s, a)$ → How good is taking action a at state s and then following policy π

Hence, if $Q_\pi(s, a_1) > Q_\pi(s, a_2)$, a_1 is a better action than a_2 at state s.

Value Functions

Function	Definition	Example Meaning
$V_\pi(s)$	Value of a state	"If I'm here, how much future reward can I expect?"
$Q_\pi(s,a)$	Value of action in a state	"If I take this action here, how much future reward can I expect?"

Value Functions

Let $\pi*$ denote for optimal policies. The corresponding state value function, called the optimal state value function $V^*(s)$:

$$V^*(s) = \max_{\pi} V_{\pi}(s) = V_{\pi^*}(s)$$

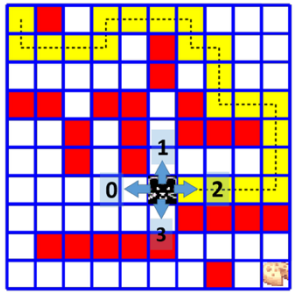
The corresponding optimal action value function, denoted Q^*:

$$Q^*(s,a) = \max_{\pi} Q_{\pi}(s,a) = Q_{\pi^*}(s,a)$$

Value Functions

We may have different policies

For example,



- $\pi_1(a_0|s) = 0.4$
- $\pi_1(a_1|s) = 0.1$
- $\pi_1(a_2|s) = 0.2$
- $\pi_1(a_3|s) = 0.3$

- $\pi_2(a_0|s) = 0.7$
- $\pi_2(a_1|s) = 0.1$
- $\pi_2(a_2|s) = 0.1$
- $\pi_2(a_3|s) = 0.1$

What is optimal policy π^*? It is the policy that maximizes the expected return (i.e., the total cumulative reward) from each state. That is,

$$\pi^* = \arg\max_{\pi} V_\pi(s), \text{ for each state } s$$

Value Functions

Let $\pi*$ denote for optimal policies. The corresponding state value function, called the **optimal state value function $V^*(s)$:**

The optimal return you can get at state s

$$V^*(s) = \max_{\pi} V_{\pi}(s) = V_{\pi^*}(s)$$

Maximum accumulated rewards/expected return state s can get

The corresponding **optimal action value function**, denoted Q^*:

The optimal return you can get if taking action a at state s

$$Q^*(s,a) = \max_{\pi} Q_{\pi}(s,a) = Q_{\pi^*}(s,a)$$

Maximum accumulated rewards/expected return can get by taking action a at state s and then following π^*

Value Functions

There can be **multiple optimal policies** that achieve the same maximum expected Return → they all share the same optimal value functions $V^*(s)$ & $Q^*(s,a)$ for all state s and state-action pairs (s,a)

Value Functions

Give an equation for $V^*(s)$ in terms of Q^*.

Give an equation for $Q^*(s, a)$ in terms of V^*.

Value Functions

Give an equation for $V^*(s)$ in terms of Q^*.

Give an equation for $Q^*(s, a)$ in terms of V^*.

The optimal return you can get at state s

$$V^*(s) = \max_a Q^*(s, a)$$

The best over all actions a of "the optimal you can get taking action a at state s"

The best possible outcome from state s, look at all the actions you can take from s, evaluate how good each action is (using $Q^*(s, a)$), and then choose the action that gives you the highest value.

Value Functions

Give an equation for $V^*(s)$ in terms of Q^*.

Give an equation for $Q^*(s,a)$ in terms of V^*.

Future rewards: The best possible rewards you can collect in the future after reaching the next state s', scaled by the discount factor γ. This discount factor tells you how much importance you give to future rewards compared to immediate ones.

Immediate reward: What you get right after taking the action

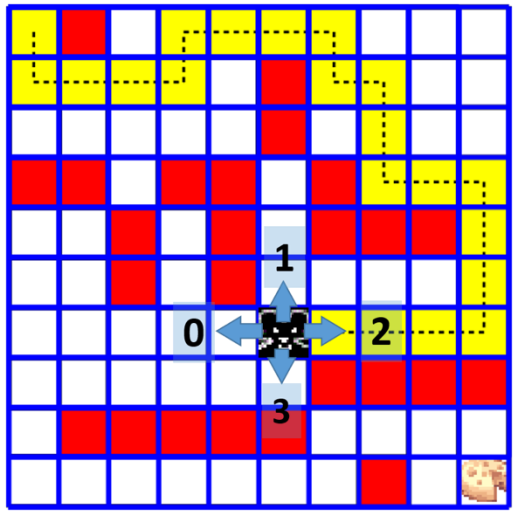
$$Q^*(s,a) = E_{s'}[r(s,a) + \gamma V^*(s')]$$

If it's a deterministic env, which means s' is determined given s and a, then, no expectation is needed.

That is, $Q^*(s,a) = r(s,a) + \gamma V^*(s')$

s' **refers to the next state**

Q-learning



if the optimal action value function Q^* is known, then in any given state s, the optimal action a^* can be found:

$$a^* = \arg\max_a Q^*(s, a)$$

For example,

$$Q^*(s, a_0) = 1$$
$$Q^*(s, a_1) = 0.3$$
$$Q^*(s, a_2) = 0.6$$
$$Q^*(s, a_3) = 0.8$$

What is a^*?

How do we get Q*?
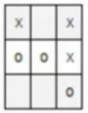
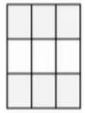
Tabular case:

- Finite state space and finite action space
- Q-table: store the Q values for each possible state and action in a lookup table

Continuous case:

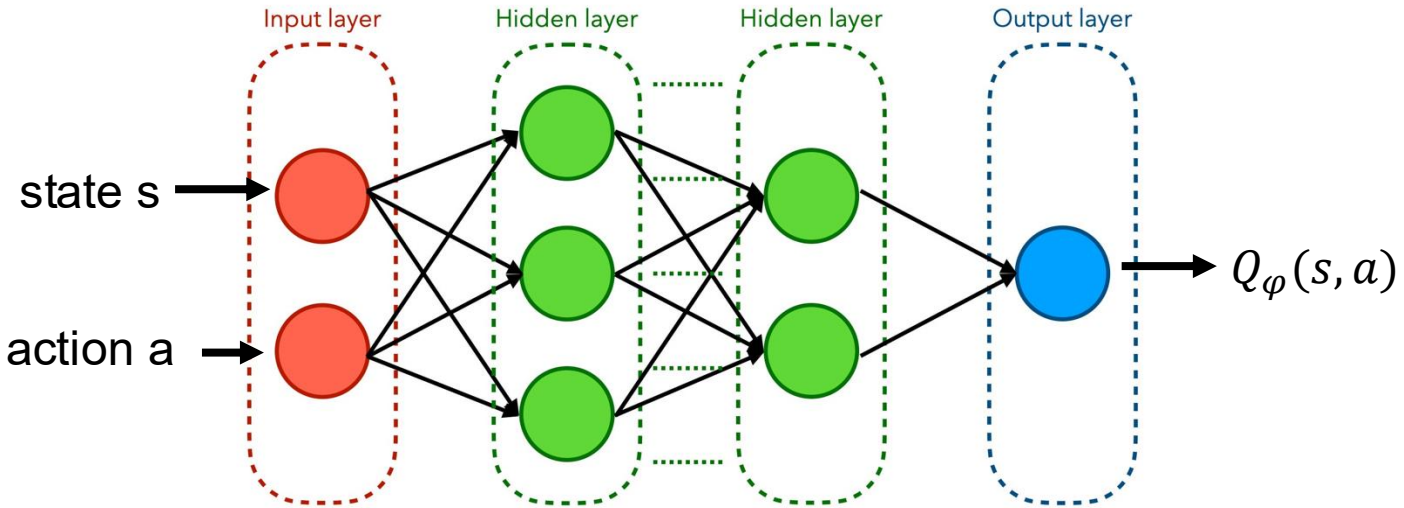
- Parameterized Q network Q_φ to approximate Q^*

Lookup Table for Q values

Game State	Top Left	Top Middle	Top Right	Middle Left	Middle Middle	Middle Right	Bottom Left	Bottom Middle	Bottom Right
	N/A	0.5	N/A	N/A	N/A	N/A	0	0	N/A
	N/A	N/A	N/A	N/A	N/A	N/A	0.5	N/A	N/A
	0.3	0.5	0.3	0.5	0.7	0.3	0.3	0.5	0.3
...	...	...	...	...	...	...	...	...	...

Q-learning: Q network

Parameterized Q network Q_φ to approximate Q^*



Mean Square Bellman Error(MSBE):

$$L(\varphi, D) = E_{(s_t,a_t,r,s_{t+1},d)\sim D}[(Q_\varphi(s_t,a_t) - y)^2]$$

Q-learning: Bellman Equation

Bellman Equation:

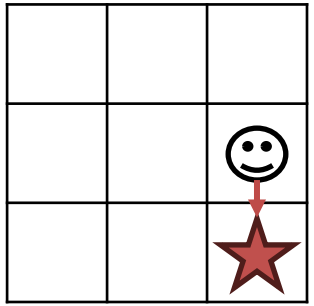
$$V^*(s_t) = E\,[r_t + \gamma V^*(s_{t+1})]$$

Fundamental relationship between the value of a state s_t and the value of the subsequent state s_{t+1}.

V*(s="goal") = 0 (episode ends here, terminal state)

V*(s_t="middle right") = $1 + \gamma \cdot V^*(s_{t+1}$ ="goal") = 1

Reaching the goal



Q-learning: Bellman Equation

Bellman Equation:

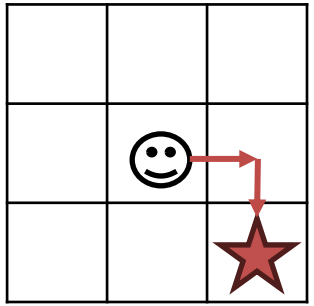
$$V^*(s_t) = E\,[r_t + \gamma V^*(s_{t+1})]$$

Fundamental relationship between the value of a state s_t and the value of the subsequent state s_{t+1}.

V*(s_t =“middle right”) = $1 + \gamma \cdot V^*(s_{t+1}$ =“goal”) = 1

V*(s_t =“middle”) = $0 + \gamma \cdot V^*(s_{t+1}$ =“middle right”)

$= 0 + \gamma \cdot 1 = \gamma$

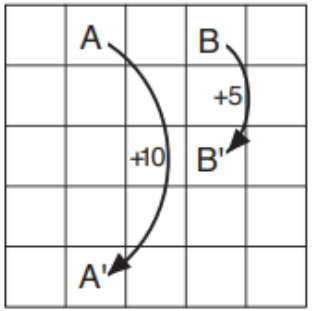


Q-learning: Bellman Equation

Grid World Example: The **cells** of the grid correspond to the **states** of the env. At each cell, four actions are possible: **up, down, left and right**. Actions that would take the agent off the grid leave its location unchanged, but also result in a reward of -1, Other actions result in a reward of 0, except those that move the agent out of the special states A and B. **From state A, all four actions yield a reward of +10 and take the agent to A' . From state B, all actions yield a reward of +5 and take the agent to B'.**

Assume $\gamma = 0.9$, optimal policy is denoted in π_*, optimal state value function is denoted by v_*

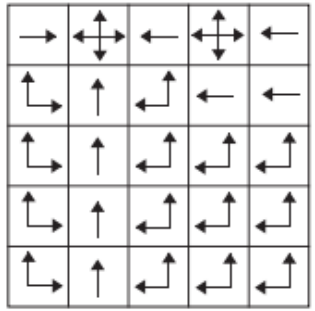
Why $v_*(A) = 24.4$?



Gridworld

22.0	24.4	22.0	19.4	17.5
19.8	22.0	19.8	17.8	16.0
17.8	19.8	17.8	16.0	14.4
16.0	17.8	16.0	14.4	13.0
14.4	16.0	14.4	13.0	11.7

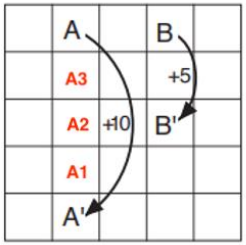
v_*



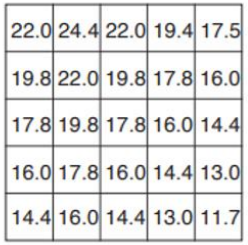
π_*

Q-learning: Bellman Equation

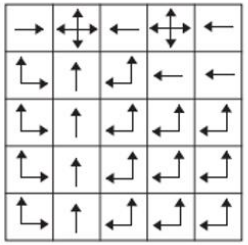
Based on the bellman equation, we know that



Gridworld



v_*


π_*

$$v_*(\mathrm{A}) = 10 + \gamma \cdot v_*(A')$$
$$v_*(\mathrm{A}') = 0 + \gamma \cdot v_*(A1)$$
$$v_*(\mathrm{A1}) = 0 + \gamma \cdot v_*(A2)$$
$$v_*(\mathrm{A2}) = 0 + \gamma \cdot v_*(A3)$$
$$v_*(\mathrm{A3}) = 0 + \gamma \cdot v_*(A)$$

$$v_*(\mathrm{A}') = 0 + \gamma \cdot (0 + \gamma \cdot v_*(A2)) = 0 + \gamma \cdot \Big(0 + \gamma \cdot \big(0 + \gamma \cdot v_*(A3)\big)\Big)$$
$$= 0 + \gamma \cdot \Big(0 + \gamma \cdot \big(0 + \gamma \cdot \big(0 + \gamma \cdot v_*(A)\big)\big)\Big) = 0 + \gamma^4 \cdot v_*(A)$$

$$v_*(A) = 10 + \gamma \cdot \big(0 + \gamma^4 \cdot v_*(A)\big) \rightarrow (1 - \gamma^5) \cdot v_*(\mathrm{A}) = 10, \text{ since } \gamma = 0.9,\ v_*(A) \approx 24.4$$

Q-learning: Bellman Equation

Bellman Equation:

$$Q^*(s_t, a_t) = E_{s_{t+1}}[r(s_t, a_t) + \gamma \max_{a_{t+1}} Q^*(s_{t+1}, a_{t+1})]$$
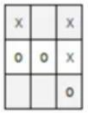
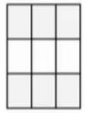
fundamental relationship between the value of a state action pair (s_t, a_t) and the value of the subsequent state action pair (s_{t+1}, a_{t+1}).

If it is a deterministic environment:

$$Q^*(s_t, a_t) = r(s_t, a_t) + \gamma \max_{a_{t+1}} Q^*(s_{t+1}, a_{t+1})$$

For tabular case, we use this relationship to update the values in the table and for continuous case, we want to train our Q network to satisfy the bellman equation.

Lookup Table for Q values

Game State	Top Left	Top Middle	Top Right	Middle Left	Middle Middle	Middle Right	Bottom Left	Bottom Middle	Bottom Right
	N/A	0.5	N/A	N/A	N/A	N/A	0	0	N/A
	N/A	N/A	N/A	N/A	N/A	N/A	0.5	N/A	N/A
	0.3	0.5	0.3	0.5	0.7	0.3	0.3	0.5	0.3
...	...	...	...	...	...	...	...	...	...

Tabular Update

For each Q value in the look-up table

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r(s_t, a_t) + \gamma \max_{a_{t+1}} Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)]$$

In this way, the Q values directly approximate optimal Q values, no matter which policy is used to collect the state and action pairs.

For correct convergence, the only requirement is that all pairs continue to be updated.

Tabular Q-learning

Randomly initialize $Q(s_t, a_t)$ for all state and action pairs

Repeat (for each episode):

Initialize s_t

Repeat (for each step of episode):

Choose action a_t at s_t using policy derived from Q (**$e.g.\ \epsilon$-greedy**)

Take action a_t, observe reward r_t, and next state s_{t+1}

Update

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r(s_t, a_t) + \gamma \max_{a_{t+1}} \boldsymbol{Q(s_{t+1}, a_{t+1})} - Q(s_t, a_t)]$$

$s_t \leftarrow s_{t+1}$

until s_t is terminal

SARSA

Randomly initialize $Q(s_t, a_t)$ for all state and action pairs

Repeat (for each episode):

Initialize s_t

Repeat (for each step of episode):

Choose action a_t at s_t using policy derived from Q ($e.g.\ \epsilon$-greedy)

Take action a_t, observe reward r_t, and next state s_{t+1}

Choose action a_{t+1} at s_{t+1} using policy derived from Q ($e.g.\ \epsilon$-greedy)

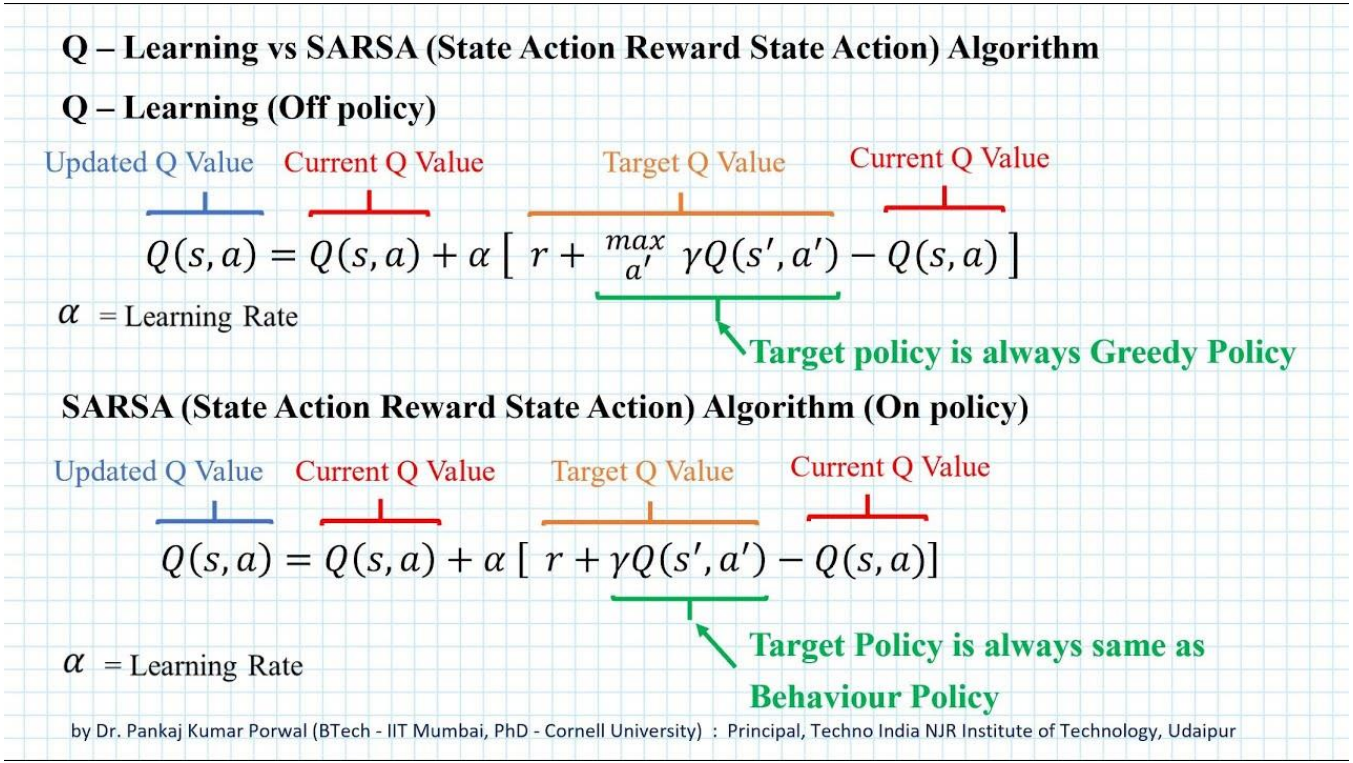
Update

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r(s_t, a_t) + \boldsymbol{\gamma\, Q(s_{t+1}, a_{t+1})} - Q(s_t, a_t)]$$

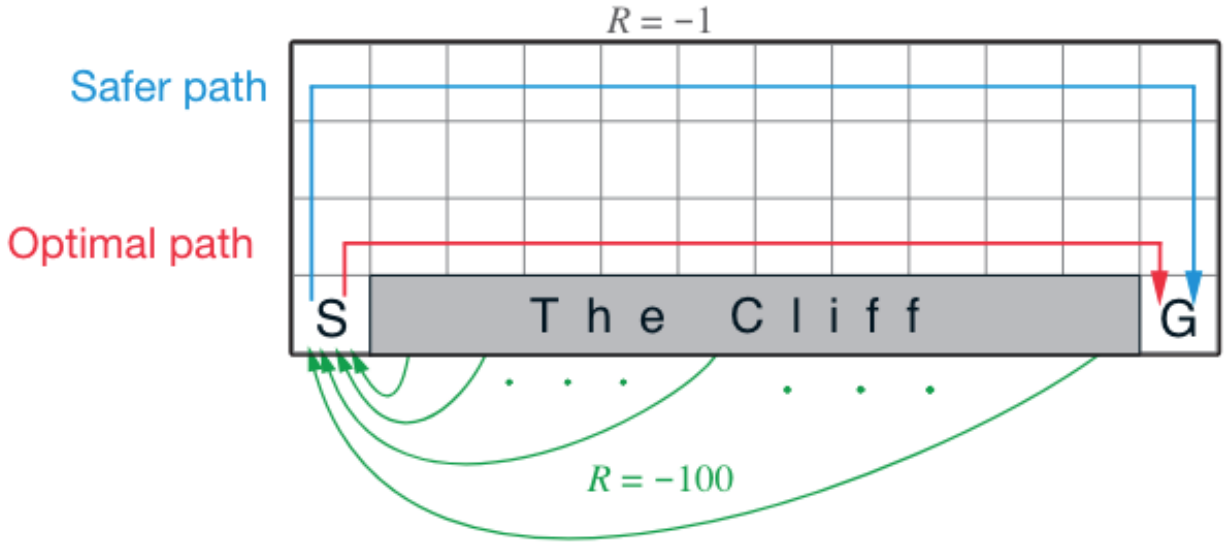
$s_t \leftarrow s_{t+1}, a_t \leftarrow a_{t+1}$

until s_t is terminal

Q-learning VS SARSA

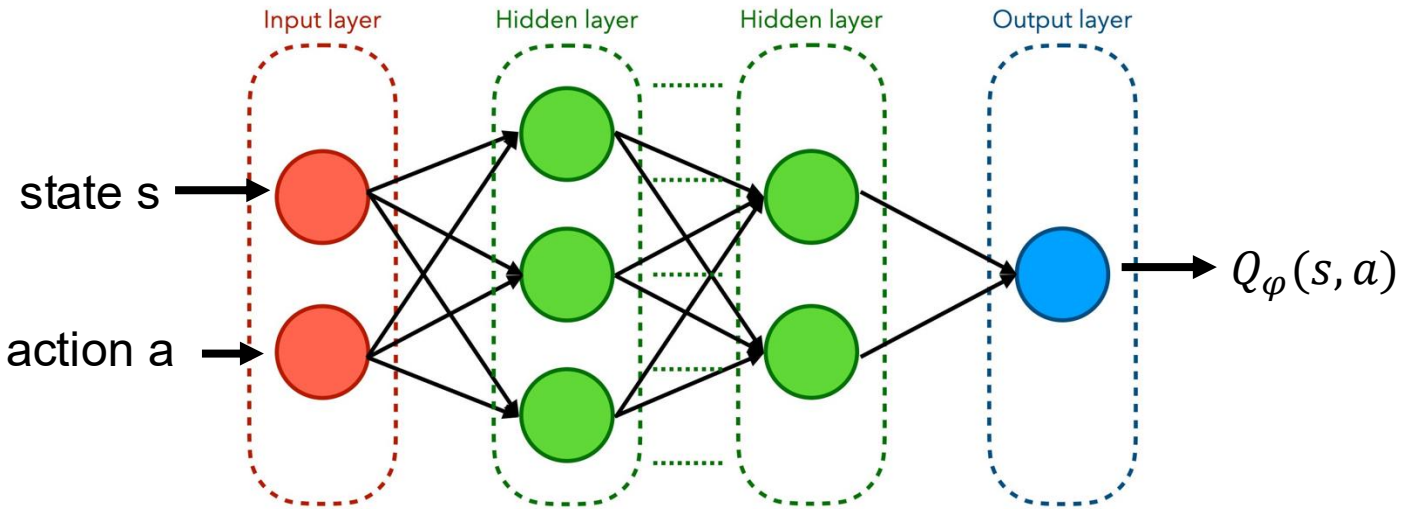


Q-learning VS SARSA



Q-learning: Q network

Parameterized Q network Q_φ to approximate Q^*



Q-learning: Training

Bellman equation target y:

$$y = r + \gamma(1-d)\max_{a_{t+1}} Q_\varphi(s_{t+1}, a_{t+1})$$

where d indicates whether s_{t+1} is a terminal state or not. That is, when s_{t+1} is a terminal state, the agent gets no additional rewards after the current state s_t.

Mean Square Bellman Error(MSBE):

$$L(\varphi, D) = E_{(s_t, a_t, r, s_{t+1}, d)\sim D}[(Q_\varphi(s_t, a_t) - y)^2]$$

where D is the collected dataset and we can use **gradient descent** to train φ:

$$\varphi \leftarrow \varphi - \alpha \cdot \nabla_\varphi L(\varphi, B)$$

where B is the mini-batch sampled from D

Q-learning: Training

Bellman equation target y:

$$y = r + \gamma(1 - d) \max_{a_{t+1}} Q_{\varphi}(s_{t+1}, a_{t+1})$$

To make the training more stable, **target Q network $Q_{\varphi'}$** to maintain a fixed target over multiple updates:

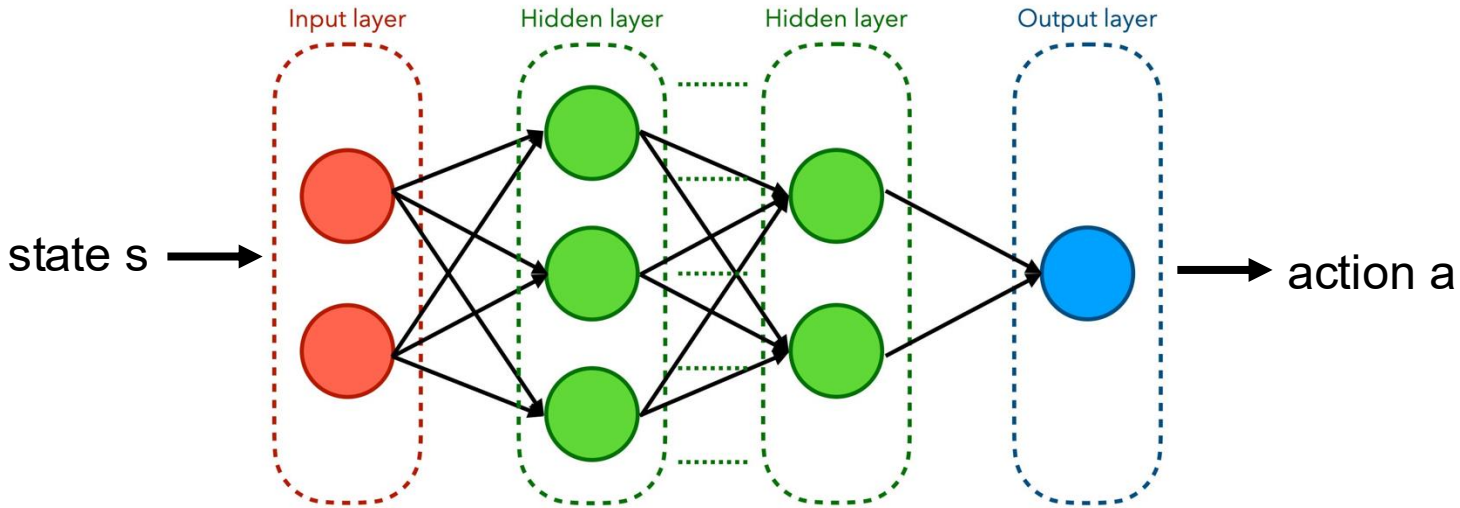
$$y = r + \gamma(1 - d) \max_{a_{t+1}} Q_{\varphi'}(s_{t+1}, a_{t+1})$$

How to determine a_{t+1} that maximize Q network?

- Discrete actions?
- Continuous actions?

Q-learning: Policy network

Policy network μ_θ where $a_t \sim \mu_\theta(s_t)$



Q-learning: Policy network

MACQUARIE University

Policy network μ_θ where $a_t \sim \mu_\theta(s_t)$

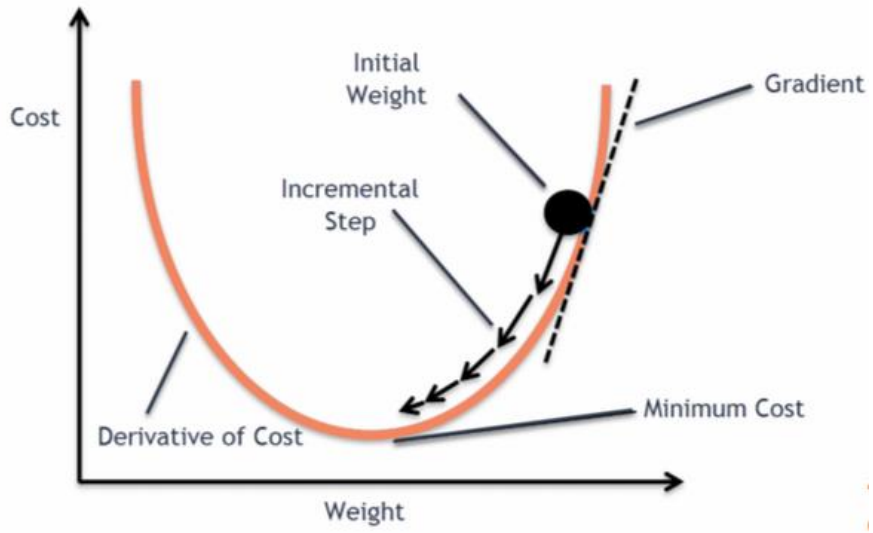
Goal of the policy network is to **output actions that maximize Q_φ** for all possible states, mathematically:

$$\theta \leftarrow arg\max_\theta E[Q_\varphi(s, \mu_\theta(s)]$$

Hence, we can use **gradient ascent** to train θ:

$$\theta \leftarrow \theta + \alpha \cdot \nabla_\theta \frac{1}{|B|} \sum_{s \in B} Q_\varphi(s, \mu_\theta(s))$$

Q-learning: Gradient Ascent



Gradient Descent

?

Gradient Ascent

Q-learning: Policy network

In MSBE, policy network μ_θ takes state s_{t+1} as input and aims to output a_{t+1} that maximizes $Q_{\varphi'}(s_{t+1}, a_{t+1})$.

Previously, Bellman equation target y:

$$y = r + \gamma(1-d)\max_{a_{t+1}} Q_\varphi(s_{t+1}, a_{t+1})$$

Mean Square Bellman Error(MSBE):

$$y = r + \gamma(1-d)\, Q_{\varphi'}(s_{t+1}, \mu_\theta(s_{t+1}))$$

$$L(\varphi, D) = E_{(s_t, a_t, r, s_{t+1}, d)\sim D}[(Q_\varphi(s_t, a_t) - y)^2]$$

Deep Deterministic Policy Gradient

Input: Q network parameters φ, target Q network parameters $\varphi' \leftarrow \varphi$, policy Network parameters θ, target policy network parameters $\theta' \leftarrow \theta$, empty replay buffer D.

Repeat:

Interact with the env using μ_θ and collect $(s_t, a_t, r, s_{t+1}, d)$ and store it in D.

if it's time to update:

Randomly sample a batch B from D.

Update φ using gradient descent: $\varphi \leftarrow \varphi - \alpha \cdot \nabla_\varphi L(\varphi, B)$

Update θ using gradient ascent: $\theta \leftarrow \theta + \alpha \cdot \nabla_\theta \frac{1}{|B|} \sum_{s \in B} Q_\varphi(s, \mu_\theta(s))$

Update target networks: $\varphi' \leftarrow \rho\varphi' + (1-\rho)\varphi$

$\theta' \leftarrow \rho\theta' + (1-\rho)\theta$

Deep Deterministic Policy Gradient



Types of RL algorithms

MACQUARIE University

- Policy gradients:
  - REINFORCE
- Value-based:
  - DQN
- Actor-critic:
  - DDPG
  - Soft Actor Critic
- Model-based RL:
  - Dyna

- Off-policy vs On-policy

Overestimation Bias

Q network is an approximation. What if Q network begins to overestimate state-action values?

For example, the optimal Q function $Q^*(s, a_i) = 0$ for all possible actions a_i at state s. However, the approximate Q values are a bit noisy:

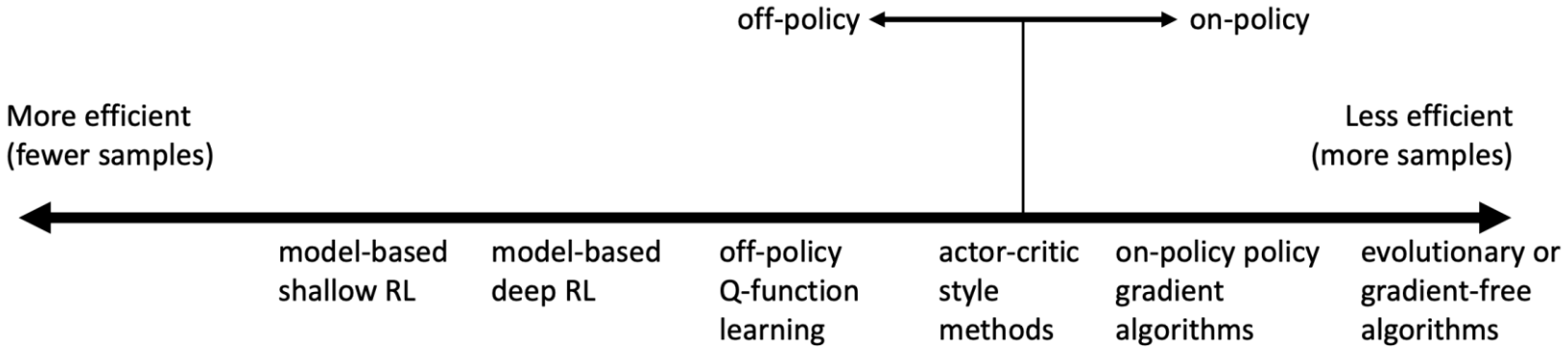
$$Q_\varphi(s, a_1) = 0.5, Q_\varphi(s, a_2) = 0.2, Q_\varphi(s, a_3) = -0.3$$

Based on the update rule, what will happen during the update?

Sample Efficiency Issue

Collect data points in real-world applications are expensive.

Wall clock time is not the same as efficiency!



How to fix the problem

Twin-delayed DDPG (TD3):

Addressing Function Approximation Error in Actor-Critic Methods

Soft-Actor-Critic (SAC):

Soft Actor-Critic Algorithms and Applications

Maxmin Q-learning:

Maxmin Q-learning: Controlling the Estimation Bias of Q-learning

REDQ:

Randomized Ensembled Double Q-Learning: Learning Fast Without a Model